

## user

通过一个实例详细讲解这三个函数

```
def normalize_text(text: str) -> str: """ Applies NFD unicode normalization, removes accents, removes punctuation, lowercases the text, and normalizes whitespace. """ # NFD unicode normalization text = unicodedata.normalize('NFD', text) # Remove accents (Nonspacing Marks) text = ''.join(c for c in text if unicodedata.category(c) != 'Mn') # Lowercase text = text.lower() # Remove punctuation text = re.sub(r'[^\w\s]', '', text) # Normalize whitespaces text = re.sub(r'\s+', ' ', text).strip() return text

def get_word_ngrams(text: str, n: int) -> set: """ Tokenizes text into word n-grams. """ words = text.split() if not words: return set() if len(words) < n: return {' '.join(words)}
```

```
return {' '.join(words[i:i+n]) for i in range(len(words) - n + 1)}
```

```
def compute_minhash_signature(ngrams: set, num_hashes: int) -> list: """ Computes a MinHash signature for a given set of n-grams. """ signature = [float('inf')] * num_hashes if not ngrams: return [0] * num_hashes
```

```
for ngram in ngrams:
    encoded_ngram = ngram.encode('utf-8')
    for i in range(num_hashes):
        # Create a deterministic, unique hash function using salt 'i'
        hasher = hashlib.md5()
        hasher.update(str(i).encode('utf-8') + b'-' + encoded_ngram)

        # Unpack the first 4 bytes into an unsigned 32-bit integer
        hash_val = struct.unpack('<I', hasher.digest()[:4])[0]
        if hash_val < signature[i]:
            signature[i] = hash_val

return signature
```

## assistant

# 通过一个完整实例详解三个文本处理函数

## 选定实例

```
raw_text = " Café is a GREAT place! Étudier à Paris. "
n = 2      # 取 2-gram
num_hashes = 3 # 签名维度
```

这个文本故意包含了重音符号（é、à）、标点（!、.）、大小写混用、多余空格，能充分触发每个函数的所有分支。

# 一、normalize\_text(text) —— 文本清洗五步走

## 第 ① 步：NFD Unicode 规范化

NFD（Normalization Form Decomposition）将预组合字符拆解为基础字符 + 组合标记：

| 原字符 | Unicode 码点   | NFD 拆解结果                       |
|-----|--------------|--------------------------------|
| é   | U+00E9（1个字符） | e (U+0065) + ◌́ (U+0301 组合锐音符) |
| à   | U+00E0（1个字符） | a (U+0061) + ◌̀ (U+0300 组合重音符) |

```
"Café" → "Cafe\u0301"    # é 拆成 e + ´
"Étudier" → "E\u0301tudier"
"à" → "a\u0300"         # à 拆成 a + `
```

## 第 ② 步：去除重音（移除类别为 Mn 的字符）

unicodedata.category(c) == 'Mn' 表示 Mark, Nonspacing（非间距标记），即组合重音符号：

```
text = ''.join(c for c in text if unicodedata.category(c) != 'Mn')
```

```
"Cafe\u0301" → "Cafe"    # U+0301 被移除
"E\u0301tudier" → "Etudier"
"a\u0300" → "a"        # U+0300 被移除
```

## 第 ③ 步：转小写

```
"Café is a GREAT place! Étudier à Paris."
→ "cafe is a great place! etudier a paris."
```

## 第 ④ 步：去除标点

正则 [^\w\s] 匹配所有非字母数字、非空白的字符（即标点）：

```
"cafe is a great place! etudier a paris."
→ "cafe is a great place etudier a paris"
```

！ 和 . 被删除。

## 第 ⑤ 步：规范化空白

```
re.sub(r'\s+', ' ', text).strip()
```

多个连续空格 → 单个空格，首尾空格去除：

```
" cafe is a great place etudier a paris "  
→ "cafe is a great place etudier a paris"
```

### ✓ 最终输出

```
normalize_text(" Café is a GREAT place! Étudier à Paris. ")  
# → "cafe is a great place etudier a paris"
```

## 二、get\_word\_ngrams(text, n) —— 提取词级 N-gram

### 输入

```
text = "cafe is a great place etudier a paris"  
n = 2
```

### 分词

```
words = text.split()  
# → ["cafe", "is", "a", "great", "place", "etudier", "a", "paris"]  
# 0 1 2 3 4 5 6 7  
# len(words) = 8
```

### 判断分支

```
len(words) = 8 ≥ n = 2 → 进入主逻辑
```

### 滑动窗口生成 2-gram

```
{words[i] + ' ' + words[i+1] for i in range(8 - 2 + 1)}  
# i 从 0 到 6，共 7 个
```

| i | 窗口         | 2-gram    |
|---|------------|-----------|
| 0 | words[0:2] | "cafe is" |

| i | 窗口         | 2-gram          |
|---|------------|-----------------|
| 1 | words[1:3] | "is a"          |
| 2 | words[2:4] | "a great"       |
| 3 | words[3:5] | "great place"   |
| 4 | words[4:6] | "place etudier" |
| 5 | words[5:7] | "etudier a"     |
| 6 | words[6:8] | "a paris"       |

✔ 返回值

```
{
  "cafe is", "is a", "a great", "great place",
  "place etudier", "etudier a", "a paris"
} # 共 7 个元素（set 类型，无序）
```

边界情况：若 `len(words) < n`（如文本只有一个词但 `n=3`），则直接返回 `{"那个词"}`；若文本为空，返回空集 `set()`。

### 三、compute\_minhash\_signature(ngrams, num\_hashes) —— MinHash 签名

#### 核心思想

用 `num_hashes` 个独立的哈希函数分别对集合中所有元素求哈希，每个哈希函数取最小值。这样得到的签名向量能近似表示集合的"指纹"。

#### 输入

```
ngrams = {"cafe is", "is a", "a great", "great place",
          "place etudier", "etudier a", "a paris"} # 7个元素
num_hashes = 3
```

#### 初始化签名

```
signature = [float('inf'), float('inf'), float('inf')]
#          hash_0      hash_1      hash_2
```

#### 对每个 n-gram × 每个哈希函数 计算

以 n-gram "cafe is" 为例：

```
encoded_ngram = b"cafe is"

# --- 哈希函数 i=0 ---
hasher = md5(b"0-cafe is")      # salt "0" + "-" + 内容
hash_val_0 = 前4字节 → 无符号32位整数

# --- 哈希函数 i=1 ---
hasher = md5(b"1-cafe is")      # salt "1" + "-" + 内容
hash_val_1 = ...

# --- 哈希函数 i=2 ---
hasher = md5(b"2-cafe is")
hash_val_2 = ...
```

`str(i).encode('utf-8') + b'-' + encoded_ngram` 中的 salt `i` 保证了同一个 n-gram 在不同哈希函数下产生不同的哈希值，模拟了"多个独立哈希函数"。

### 逐元素更新最小值（伪代码演示）

假设遍历 7 个 n-gram 后，各哈希函数看到的最小值为：

|                 | hash_0 (salt "0") | hash_1 (salt "1") | hash_2 (salt "2") |
|-----------------|-------------------|-------------------|-------------------|
| "cafe is"       | 2,841,003         | 1,293,847         | 3,102,445         |
| "is a"          | 1,002,394         | 982,113           | 2,847,001         |
| "a great"       | 3,291,445         | 2,104,992         | 583,221           |
| "great place"   | 871,220           | 3,401,228         | 1,994,330         |
| "place etudier" | 2,100,384         | 774,029           | 2,331,887         |
| "etudier a"     | 1,553,001         | 2,881,443         | 910,224           |
| "a paris"       | 445,112           | 1,003,221         | 3,445,001         |
| 取 min ↓         | 445,112           | 774,029           | 583,221           |

```
signature = [445112, 774029, 583221]
```

### ✔ 返回值

```
compute_minhash_signature(ngrams, 3)
# → [445112, 774029, 583221] # 具体数值取决于 MD5 实际输出
```

边界情况：若 `ngrams` 为空集，直接返回 `[0, 0, 0]`。

## 四、三函数串联：完整 Pipeline

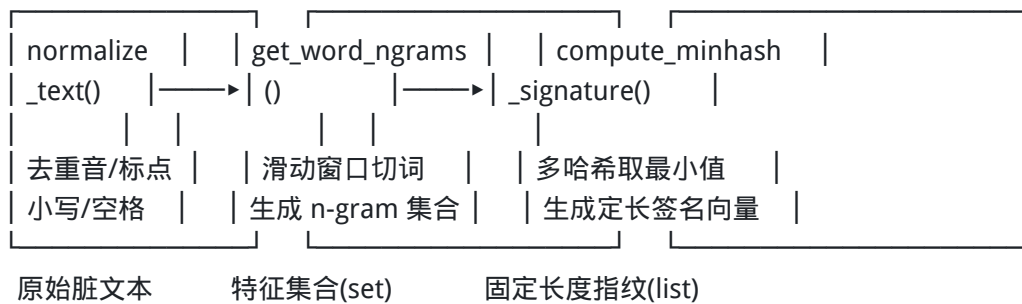
```
import unicodedata, re, hashlib, struct

raw = " Café is a GREAT place! Étudier à Paris. "

step1 = normalize_text(raw)
# "cafe is a great place etudier a paris"

step2 = get_word_ngrams(step1, n=2)
# {"cafe is", "is a", "a great", "great place", "place etudier", "etudier a", "a paris"}

step3 = compute_minhash_signature(step2, num_hashes=3)
# [445112, 774029, 583221] ← 这就是该文本的 MinHash 指纹
```



实际用途：两篇文档各自生成 MinHash 签名后，比较签名中相同位置相等的比例，即可近似估算两文档的 Jaccard 相似度，常用于大规模文本去重和近似匹配。

### user

接下来详细讲解主函数 `def deduplicate_documents( input_paths: list, num_hashes: int, num_bands: int, ngram_size: int, jaccard_threshold: float, output_dir: str ):` Deduplicates a list of files using MinHash and LSH, and writes the retained files to the output directory.

```
Args:
    input_paths: List of file paths to process.
    num_hashes: Length of the MinHash signature.
    num_bands: Number of bands to divide the signature into for LSH.
    ngram_size: The 'n' in n-gram length (in words).
    jaccard_threshold: The true Jaccard similarity threshold above which documents
        are considered duplicates.
    output_dir: Directory where the deduplicated files will be saved.
"""
if num_hashes % num_bands != 0:
    raise ValueError("num_hashes must be evenly divisible by num_bands.")

os.makedirs(output_dir, exist_ok=True)
rows_per_band = num_hashes // num_bands

doc_ngrams = {}
signatures = {}
```

# 1. Read files, normalize, extract n-grams, and compute MinHash signatures

for path in input\_paths:

with open(path, 'r', encoding='utf-8') as f:

text = f.read()

norm\_text = normalize\_text(text)

ngrams = get\_word\_ngrams(norm\_text, ngram\_size)

doc\_ngrams[path] = ngrams

signatures[path] = compute\_minhash\_signature(ngrams, num\_hashes)

# 2. Locality Sensitive Hashing (LSH) to identify candidate pairs

lsh\_buckets = defaultdict(list)

candidates = set()

for path, sig in signatures.items():

for band\_idx in range(num\_bands):

start = band\_idx \* rows\_per\_band

end = start + rows\_per\_band

# A band is represented as a tuple of hash values

band\_tuple = tuple(sig[start:end])

# Add a band prefix so bands don't collide across different parts of the signature

bucket\_key = (band\_idx, band\_tuple)

lsh\_buckets[bucket\_key].append(path)

# Generate candidate pairs from buckets that share at least one band

for bucket\_key, docs in lsh\_buckets.items():

if len(docs) > 1:

for u, v in itertools.combinations(sorted(docs), 2):

candidates.add((u, v))

# 3. Filter candidates by true n-gram Jaccard similarity

duplicate\_edges = []

for u, v in candidates:

set\_u, set\_v = doc\_ngrams[u], doc\_ngrams[v]

intersection = len(set\_u & set\_v)

union = len(set\_u | set\_v)

sim = intersection / union if union > 0 else 1.0

if sim >= jaccard\_threshold:

duplicate\_edges.append((u, v))

# 4. Cluster duplicates into Connected Components

# Treat paths as nodes and true duplicate relationships as undirected edges

adj = {path: [] for path in input\_paths}

for u, v in duplicate\_edges:

adj[u].append(v)

adj[v].append(u)

visited = set()

components = []

for path in input\_paths:

if path not in visited:

```
comp = []
stack = [path]
visited.add(path)

while stack:
    curr = stack.pop()
    comp.append(curr)
    for neighbor in adj[curr]:
        if neighbor not in visited:
            visited.add(neighbor)
            stack.append(neighbor)
    components.append(comp)

# 5. Randomly retain exactly one document from each component/cluster
retained_paths = []
for comp in components:
    # For non-duplicates, the component size is 1, so the document is inherently kept
    retained = random.choice(comp)
    retained_paths.append(retained)

# 6. Write out the retained files to the output directory
for path in retained_paths:
    filename = os.path.basename(path)
    out_path = os.path.join(output_dir, filename)
    shutil.copy2(path, out_path)
```

assistant

# 主函数 deduplicate\_documents 详解

## 构造一个完整实例

假设我们有 5 个文本文件需要去重：

| 文件路径       | 内容（简化）                                                     |
|------------|------------------------------------------------------------|
| docs/a.txt | "The quick brown fox jumps over the lazy dog"              |
| docs/b.txt | "The quick brown fox jumps over the lazy dog" （与 a 完全相同）   |
| docs/c.txt | "The quick brown fox jumps over the lazy cat" （与 a 仅一词之差）  |
| docs/d.txt | "Machine learning is transforming the world" （完全不同）        |
| docs/e.txt | "Deep learning is transforming the world today" （与 d 高度相似） |

调用参数：

```
deduplicate_documents(
    input_paths = ["docs/a.txt", "docs/b.txt", "docs/c.txt", "docs/d.txt", "docs/e.txt"],
    num_hashes = 6, # MinHash 签名长度
```

```
num_bands      = 3,      # LSH 分段数
ngram_size     = 2,      # 2-gram
jaccard_threshold= 0.5,   # Jaccard  $\geq 0.5$  视为重复
output_dir     = "output/"
)
```

## 〇、参数校验与初始化

```
if num_hashes % num_bands != 0:
    raise ValueError("num_hashes must be evenly divisible by num_bands.")
```

为什么必须整除？因为 LSH 要把签名均匀切段。  $6 \% 3 == 0$  ✓，每段  $\text{rows\_per\_band} = 6 // 3 = 2$  行。

```
os.makedirs(output_dir, exist_ok=True) # 创建输出目录（已存在则跳过）
rows_per_band = num_hashes // num_bands # = 2
```

两个核心字典：

```
doc_ngrams = {} # path → n-gram 集合（后面算真实 Jaccard 用）
signatures = {} # path → MinHash 签名向量（LSH 分桶用）
```

## 第一步：读取 → 清洗 → 提取 N-gram → 计算签名

```
for path in input_paths:
    text = open(path).read()
    norm_text = normalize_text(text) # 清洗
    ngrams = get_word_ngrams(norm_text, 2) # 2-gram
    doc_ngrams[path] = ngrams
    signatures[path] = compute_minhash_signature(ngrams, 6)
```

以 docs/a.txt 为例，走一遍上节讲过的三步：

```
"The quick brown fox jumps over the lazy dog"
↓ normalize_text（已无重音/标点，仅小写）
"the quick brown fox jumps over the lazy dog"
↓ get_word_ngrams(text, 2)
{"the quick", "quick brown", "brown fox", "fox jumps",
 "jumps over", "over the", "the lazy", "lazy dog"} # 8 个 2-gram
↓ compute_minhash_signature(ngrams, 6)
[1823401, 944210, 3310077, 512834, 2741990, 88734] # 6 维签名（示意值）
```

全部 5 个文件处理完后：

```
signatures = {
    "docs/a.txt": [1823401, 944210, 3310077, 512834, 2741990, 88734],
    "docs/b.txt": [1823401, 944210, 3310077, 512834, 2741990, 88734], # 与 a 完全相同
    "docs/c.txt": [1823401, 200133, 3310077, 512834, 440012, 88734], # 与 a 高度相似
    "docs/d.txt": [77120, 3390211, 100284, 2984510, 1550023, 3102847],
    "docs/e.txt": [77120, 1200344, 100284, 2984510, 1550023, 891002], # 与 d 较相似
}
```

注意：a 和 b 内容完全相同 → 签名完全一致；a 和 c 仅一词之差 → 签名大部分相同。

## 第二步：LSH 分桶 —— 找出候选重复对

### 2.1 签名切段 (Banding)

把 6 维签名按 `rows_per_band=2` 切成 3 个 band：

签名: [h0, h1, h2, h3, h4, h5]

band 0   band 1   band 2

对 docs/a.txt：

| band_idx | start:end | band_tuple        |
|----------|-----------|-------------------|
| 0        | [0:2]     | (1823401, 944210) |
| 1        | [2:4]     | (3310077, 512834) |
| 2        | [4:6]     | (2741990, 88734)  |

### 2.2 投入哈希桶

```
bucket_key = (band_idx, band_tuple) # 带 band 编号，防止跨 band 碰撞
lsh_buckets[bucket_key].append(path)
```

为什么加 `band_idx`？如果 band 0 的值恰好等于 band 1 的值，不加前缀会把它们误认为同一个桶。

遍历所有 5 个文件后，桶的内容（示意）：

```
lsh_buckets = {
    (0, (1823401, 944210)): ["docs/a.txt", "docs/b.txt"],      # a,b 落入同桶
    (0, (1823401, 200133)): ["docs/c.txt"],                  # c 单独
    (0, (77120, 3390211)): ["docs/d.txt"],
    (0, (77120, 1200344)): ["docs/e.txt"],

    (1, (3310077, 512834)): ["docs/a.txt", "docs/b.txt", "docs/c.txt"], # a,b,c 同桶！
}
```

```
(1, (100284, 2984510)): ["docs/d.txt", "docs/e.txt"], # d,e 同桶 !

(2, (2741990, 88734)): ["docs/a.txt", "docs/b.txt", "docs/c.txt"],
(2, (1550023, 3102847)): ["docs/d.txt"],
(2, (1550023, 891002)): ["docs/e.txt"],
}
```

## 2.3 从桶中生成候选对

```
for bucket_key, docs in lsh_buckets.items():
    if len(docs) > 1: # 桶内 ≥ 2 个文档才有候选对
        for u, v in itertools.combinations(sorted(docs), 2):
            candidates.add((u, v))
```

| 桶                     | 文档      | 产生的候选对                 |
|-----------------------|---------|------------------------|
| (0, (1823401,944210)) | a, b    | (a, b)                 |
| (1, (3310077,512834)) | a, b, c | (a, b), (a, c), (b, c) |
| (1, (100284,2984510)) | d, e    | (d, e)                 |
| (2, (2741990,88734))  | a, b, c | (a, b), (a, c), (b, c) |

去重后：

```
candidates = {
    ("docs/a.txt", "docs/b.txt"),
    ("docs/a.txt", "docs/c.txt"),
    ("docs/b.txt", "docs/c.txt"),
    ("docs/d.txt", "docs/e.txt"),
}
```

🔑 LSH 的核心价值：5 个文档本来有  $C(5,2)=10$  对需要比较，LSH 把候选对缩减到 4 对，避免了  $O(n^2)$  全量比较。文档量到百万级时，这个加速至关重要。

## 第三步：精确验证 —— 真实 Jaccard 相似度

LSH 只是粗筛（可能有假阳性），所以要用真实的 n-gram 集合精确计算：

```
for u, v in candidates:
    set_u, set_v = doc_ngrams[u], doc_ngrams[v]
    intersection = len(set_u & set_v) # 交集大小
    union = len(set_u | set_v) # 并集大小
    sim = intersection / union if union > 0 else 1.0
```

```
if sim >= jaccard_threshold: #  $\geq 0.5$   
    duplicate_edges.append((u, v))
```

逐对计算：

(a, b) : 完全相同

a 的 2-gram: {the quick, quick brown, brown fox, fox jumps,  
jumps over, over the, the lazy, lazy dog} # 8个  
b 的 2-gram: 完全相同 # 8个

交集 = 8, 并集 = 8

Jaccard =  $8/8 = 1.0 \geq 0.5$  ✓ → 加入 duplicate\_edges

(a, c) : 仅 "dog" → "cat" 一词之差

a: {the quick, quick brown, brown fox, fox jumps,  
jumps over, over the, the lazy, lazy dog} # 8个  
c: {the quick, quick brown, brown fox, fox jumps,  
jumps over, over the, the lazy, lazy cat} # 8个

交集 = 7 ( 除 lazy dog / lazy cat )

并集 = 9 ( 7 + lazy dog + lazy cat )

Jaccard =  $7/9 \approx 0.778 \geq 0.5$  ✓ → 加入

(b, c) : 同理

Jaccard =  $7/9 \approx 0.778 \geq 0.5$  ✓ → 加入

(d, e)

d: {machine learning, learning is, is transforming,  
transforming the, the world} # 5个  
e: {deep learning, learning is, is transforming,  
transforming the, the world, world today} # 6个

交集 = 4 ( learning is, is transforming, transforming the, the world )

并集 = 7

Jaccard =  $4/7 \approx 0.571 \geq 0.5$  ✓ → 加入

```
duplicate_edges = [  
    ("docs/a.txt", "docs/b.txt"),  
    ("docs/a.txt", "docs/c.txt"),  
    ("docs/b.txt", "docs/c.txt"),  
    ("docs/d.txt", "docs/e.txt"),  
]
```

## 第四步：连通分量聚类（Union-Find 的 DFS 版）

### 4.1 建图

把每个文件看作节点，重复关系看作无向边：

```
adj = {path: [] for path in input_paths}
for u, v in duplicate_edges:
    adj[u].append(v)
    adj[v].append(u)
```

```
adj = {
    "docs/a.txt": ["docs/b.txt", "docs/c.txt"],
    "docs/b.txt": ["docs/a.txt", "docs/c.txt"],
    "docs/c.txt": ["docs/a.txt", "docs/b.txt"],
    "docs/d.txt": ["docs/e.txt"],
    "docs/e.txt": ["docs/d.txt"],
}
```

图示：

```
a — b      d — e
|  /
c
【分量1】    【分量2】
```

### 4.2 DFS 遍历找连通分量

```
visited = set()
components = []

for path in input_paths:
    if path not in visited:
        comp = []
        stack = [path]
        visited.add(path)
        while stack:
            curr = stack.pop()
            comp.append(curr)
            for neighbor in adj[curr]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    stack.append(neighbor)
        components.append(comp)
```

执行轨迹：

| 轮次  | 起点      | DFS 访问顺序                            | 分量        |
|-----|---------|-------------------------------------|-----------|
| 1   | a.txt   | a → c → b ( 或 a → b → c, 取决于邻接表顺序 ) | [a, c, b] |
| 2   | d.txt   | d → e                               | [d, e]    |
| 3~5 | b, c, e | 已在 visited 中, 跳过                    | —         |

```
components = [  
    ["docs/a.txt", "docs/c.txt", "docs/b.txt"], # 分量1: 3个互为重复  
    ["docs/d.txt", "docs/e.txt"],           # 分量2: 2个互为重复  
]
```

🔑 为什么要聚类？因为重复关系具有传递性： $a \approx b, b \approx c \rightarrow a, b, c$  应归为同一组，只保留一篇。如果不聚类，可能出现"删了 a 但 b、c 都留着"的不一致结果。

## 第五步：每个分量随机保留一篇

```
retained_paths = []  
for comp in components:  
    retained = random.choice(comp) # 随机选一个  
    retained_paths.append(retained)
```

| 分量        | 候选  | 随机选中（示例）   |
|-----------|-----|------------|
| [a, c, b] | 三选一 | docs/b.txt |
| [d, e]    | 二选一 | docs/d.txt |

```
retained_paths = ["docs/b.txt", "docs/d.txt"]
```

注意：如果某文档与任何其他文档都不重复，它自己就是一个大小为 1 的分量，`random.choice` 必然选中它自身 → 不会误删。

## 第六步：输出保留文件

```
for path in retained_paths:  
    filename = os.path.basename(path)      # "b.txt"  
    out_path = os.path.join(output_dir, filename) # "output/b.txt"  
    shutil.copy2(path, out_path)           # 复制（保留元数据）
```

最终 `output/` 目录：

output/

└─ b.txt ← 代表 {a, b, c} 这组重复  
└─ d.txt ← 代表 {d, e} 这组重复

5 篇 → 2 篇，去除了 3 篇重复。

## 全流程总览

Step 1: 读取 & 特征提取

file → normalize\_text → get\_word\_ngrams → compute\_minhash\_sig

得到: doc\_ngrams{} + signatures{}  
▼

Step 2: LSH 粗筛 ( 候选对生成 )

签名切 band → 投桶 → 同桶文档两两配对

$O(n^2)$  比较 → 缩减为  $O(\text{候选对数})$   
▼

Step 3: 精确验证

对每个候选对计算真实 Jaccard(n-gram 集合)

$\geq \text{threshold}$  → 确认为重复边  
▼

Step 4: 连通分量聚类

重复边 → 无向图 → DFS 找连通分量

传递性:  $a \approx b, b \approx c \rightarrow \{a, b, c\}$  归为一组  
▼

Step 5: 每组随机保留一篇

random.choice(component)  
▼

Step 6: 写出结果

shutil.copy2 → output\_dir

## 关键参数对效果的影响

| 参数                | 增大效果                              | 减小效果                         |
|-------------------|-----------------------------------|------------------------------|
| num_hashes        | 签名更精确，Jaccard 估计更准，但计算更慢          | 更快但更粗糙                       |
| num_bands         | 每 band 行数减少 → 召回率 ↑（更容易进同桶），但假阳性↑ | 每 band 行数增加 → 精确率 ↑，但可能漏掉相似对 |
| ngram_size        | 更大 n → 更细粒度，对词序更敏感                | 更小 n → 更粗粒度，容错更高             |
| jaccard_threshold | 更严格，只去除高度重复                       | 更宽松，中度相似也会被去除                |

num\_bands 与 rows\_per\_band 的权衡是 LSH 的核心调参点：两篇文档在某个 band 完全匹配的概率为  $s^r$ （ $s$  为真实 Jaccard， $r$  为每 band 行数）。至少一个 band 匹配的概率为  $1-(1-s^r)^b$ （ $b$  为 band 数）。这个 S 曲线的拐点大约在  $s^* = (1/b)^{1/r}$ ，即阈值附近的相似对最容易被捕获。